DeltaNet derives its name from the core principle of updating weights based on the "delta" between the prediction $S_{t-1}k_t$ and target v_t

$$\mathcal{L}_t(S) = \frac{1}{2}||Sk_t - v_t||^2$$

Train S such that the information retrieved using k_t is as close as possible to v_t at each timestep.

$$\begin{aligned} S_t &= S_{t-1} - \beta_t \nabla_S \mathcal{L}_t(S_{t-1}) \\ &= S_{t-1} - \beta_t (S_{t-1}k_t - v_t)k_t^T \\ &= S_{t-1}(\mathrm{I} - \beta_t k_t k_t^T) + \beta_t v_t k_t^T \end{aligned}$$

Let $M_t = \mathrm{I} - \beta_t k_t k_t^T$ and $N_t = \beta_t v_t k_t^T$

$$S_t = S_{t-1}M_t + N_t$$

$(M_i, N_i) \otimes (M_j, N_j) = (M_i M_j, N_i M_j + N_j)$ is an associative operator. We can therefore compute all states in parallel and play the "chunk" trick to save memory.

Unroll the recurrence:

$$S_t = \sum_{i=0}^{t} N_i \prod_{j=i+1}^{t} M_j$$

Proof by induction:

$$\begin{aligned} P_a^b &= P_a^{b-1}(\mathrm{I} - \beta_b k_b k_b^T) = \left(\mathrm{I} - \sum_{t=a}^{b-1} w_t k_t^T\right)(\mathrm{I} - \beta_b k_b k_b^T) \\ &= \mathrm{I} - \sum_{t=a}^{b-1} w_t k_t^T - \underbrace{(\beta_b k_b - \beta_b \sum_{t=a}^{b-1}(w_t(k_t^T k_b)))}_{w_b} k_b^T \\ &= \mathrm{I} - \sum_{t=a}^{b} w_t k_t^T \end{aligned}$$

Let $P_a^b = \prod_{t=a}^{b} M_t = \prod_{t=a}^{b}(\mathrm{I} - \beta_t k_t k_t^T)$

Its compact representation is:

$\mathrm{I} - \sum_{t=a}^{b} w_t k_t^T$ where $w_t = \beta_t k_t - \beta_t \sum_{n=a}^{t-1}(w_n(k_n^T k_t))$

Similarly, let $H_a^b = \sum_{t=a}^{b} N_t P_{t+1}^b$ and its compact representation is:

$\sum_{t=a}^{b} u_t k_t^T$ where $u_t = \beta_t v_t - \beta_t \sum_{n=a}^{t-1}(u_n(k_n^T k_t))$

Once w_t and u_t have been constructed, the rest can proceed as in ordinary linear attention.

A graph theory trick is then applied to transform the recurrent constructions of w_t and u_t into a hardware-efficient form. Taking w_t as an example, the trick is to consider its recurrent form as an adjacency matrix A (a weighted directed graph) such that nodes represent time steps and edges connect step n to t ($n < t$) with weight $-\beta_t k_i^T k_t$. A chunk of A can be computed efficiently: $A_a^b = triu(-K_a^{b\,T} K_a^b diag(\beta_a^b), 1)$.

The theory also tells us that each entry $[i, j]$ of $(\mathrm{I} - A)^{-1}$ gives the sum of weights of all possible paths from i to j. Taking advantage of this accumulated feature, a chunk of W can be formulated as $W_a^b = K_a^b diag(\beta_a^b)(\mathrm{I} - A_b^b)^{-1}$. The inversion of upper triangular matrices can be solved efficiently using backward substitution.

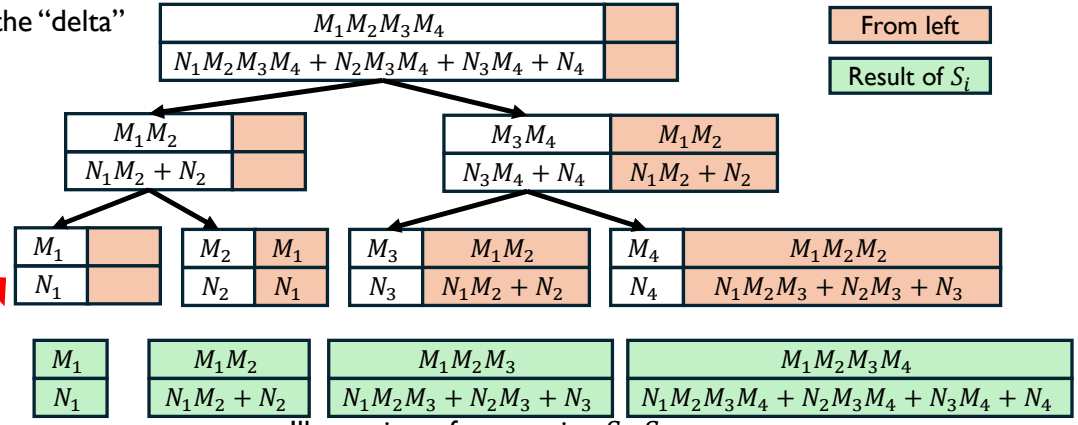
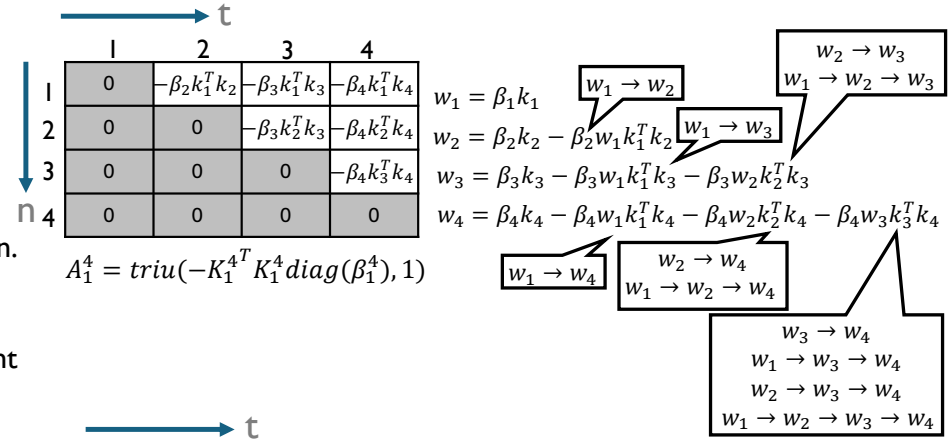
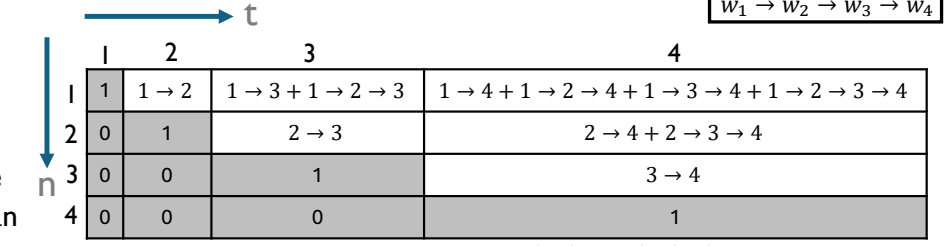


Illustration of computing S_1-S_4



$$A_1^4 = triu(-K_1^{4\,T} K_1^4 diag(\beta_1^4), 1)$$



$$(\mathrm{I} - A_1^4)^{-1} = \mathrm{I} + A_1^4 + A_1^4A_1^4 + A_1^4A_1^4A_1^4$$